UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 3

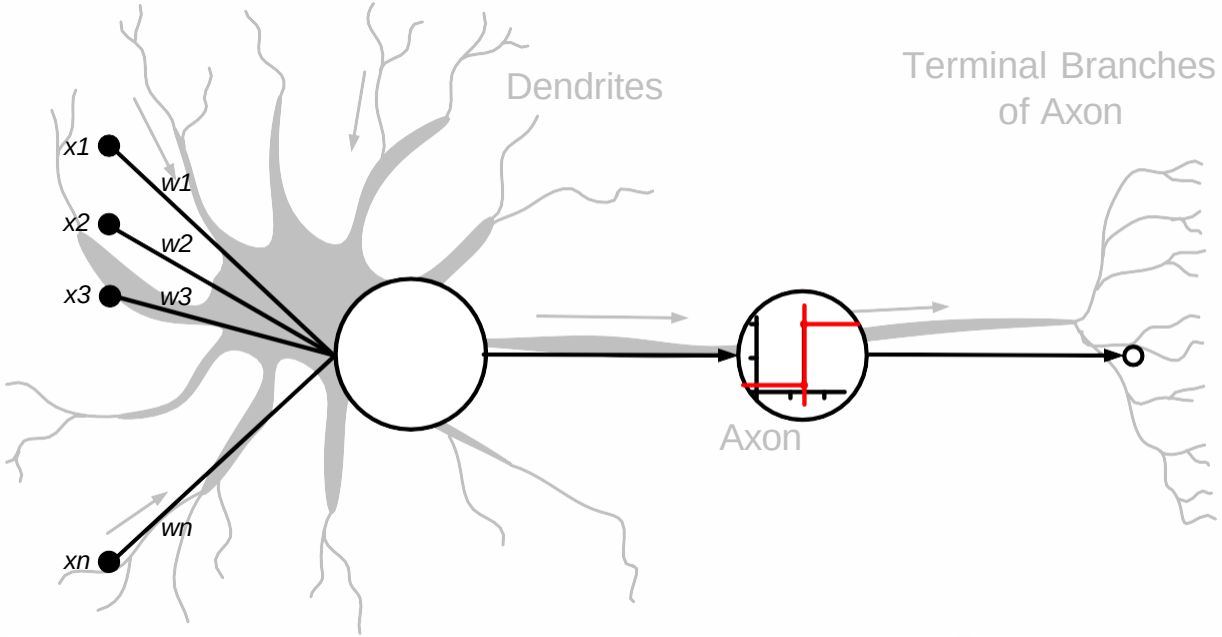
Perceptron and Multilayer Perceptron

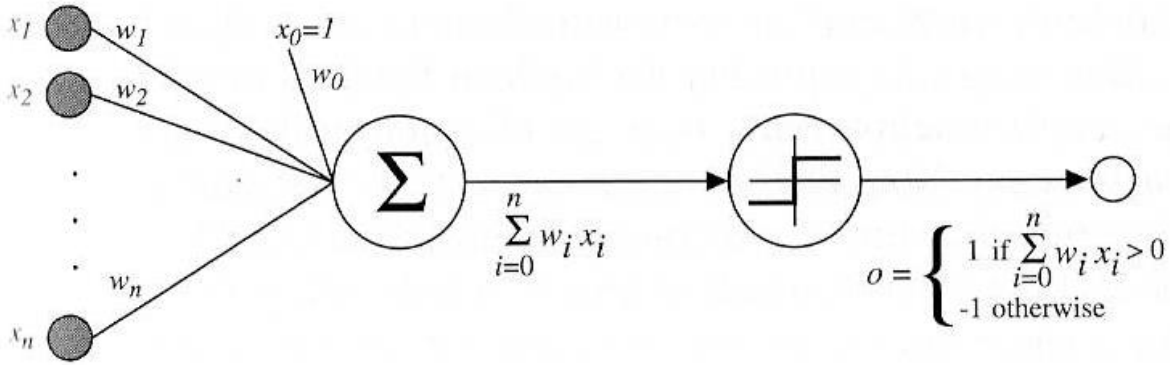
Dr. Haza Nuzly Abdull Hamed
Assoc. Prof. Dr. Siti Zaiton Mohd Hashim

Introduction to Perceptron

- The aim of the perceptron is to classify a set of inputs into one of two categories (usually 1 or 0).
- If the inputs are in the form of a grid, a perceptron can be used to recognize visual images of shapes.
- The perceptron usually uses a step function, which returns 1 if the weighted sum of inputs exceeds a threshold, and 0 otherwise.

Perceptron structure





Learning Rules

1. Perceptron Learning Rule
   - The perceptron training algorithm for classification tasks.

2. Delta Rules
   - For continuous activation function
   - The aim of the delta rule is to minimize the error over all training patterns

Perceptron Training Algorithm

1. Set initial weights w_1, w_2,…, w_n and threshold θ to random numbers. Normally [-0.5,0.5]

2. Calculate output:

$$X = \sum_{i=1}^{n} w_i x_i$$

3. Apply activation functions

$$Y = \begin{cases} 1 & for\ X > \theta \\ 0 & for\ X \leq \theta \end{cases}$$

Perceptron Training Algorithm

4. An item of training data is presented. If the perceptron mis-classifies it, the weights are modified according to the following:

$$w_i \leftarrow w_i + (a \times x_i \times (t - o))$$

where *t* is the target output for the training example, *o* is the output generated by the preceptron and *a* is the learning rate, between 0 and 1 (usually small such as 0.1)

5. Cycle through training examples until successfully classify all examples

Linear and Non Linear Separable

1	1	1
0	0	1
OR	0	1

1	0	1
0	0	0
AND	0	1

1	1	0
0	1	0
NOT	0	1

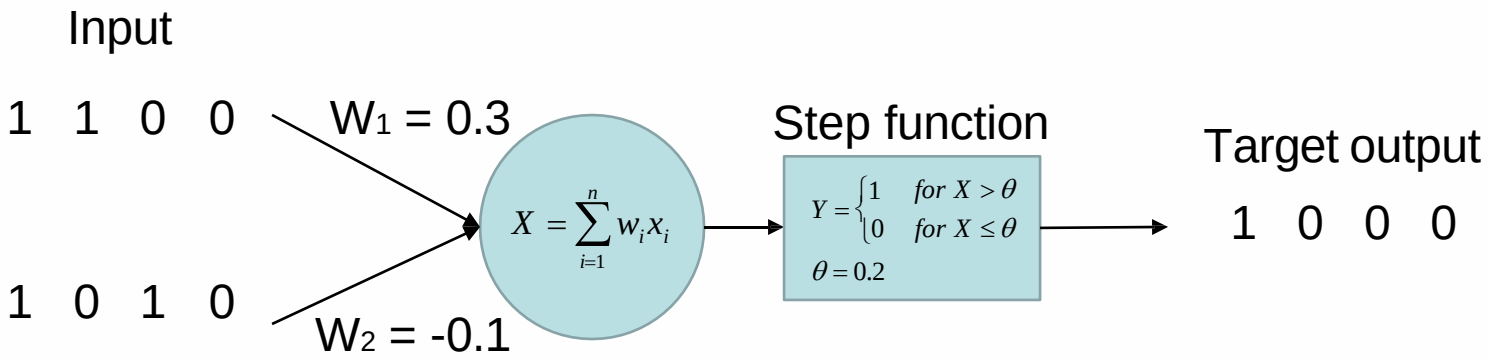
OR, AND and NOT are linearly separable Boolean Functions

1	1	0
0	0	1
XOR	0	1

XOR is not linearly separable

Perceptron learning example: AND

1	0	1
0	0	0
AND	0	1



Pattern 1: (0x0.3)+(0x-0.1) = 0, Less than 0.2, output = 0 (target 0)
Pattern 2: (0x0.3)+(1x-0.1) = -0.1, Less than 0.2, output = 0 (target 0)
Pattern 3: (1x0.3)+(0x-0.1) = 0.3, Greater than 0.2, output = 1 (target 0)
Weight update: $w_i \leftarrow w_i + (a \times x_i \times (t - o))$
w_1 = 0.3 + (0.1 x 1 x (0-1) = 0.2
w_2 = -0.1 + (0.1 x 0 x (0-1) = -0.1
Pattern 4: (1x0.2)+(1x-0.1) = 0.1, Less than 0.2, output = 0 (target 1)
Weight update:…..

Perceptron learning example: AND

Epoch	Inputs		Desired output	Initial weights		Actual output	Error	Final weights	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
1	0	0	0	0.3	–0.1	0	0	0.3	–0.1
	0	1	0	0.3	–0.1	0	0	0.3	–0.1
	1	0	0	0.3	–0.1	1	–1	0.2	–0.1
	1	1	1	0.2	–0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	–1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	–1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	–1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Threshold: $\theta = 0.2$; learning rate: $\alpha = 0.1$

Perceptron Assignment:

Using Lab Exercise 1 code, solve OR and NOT problem using perceptron. Select your own threshold and learning rate values.

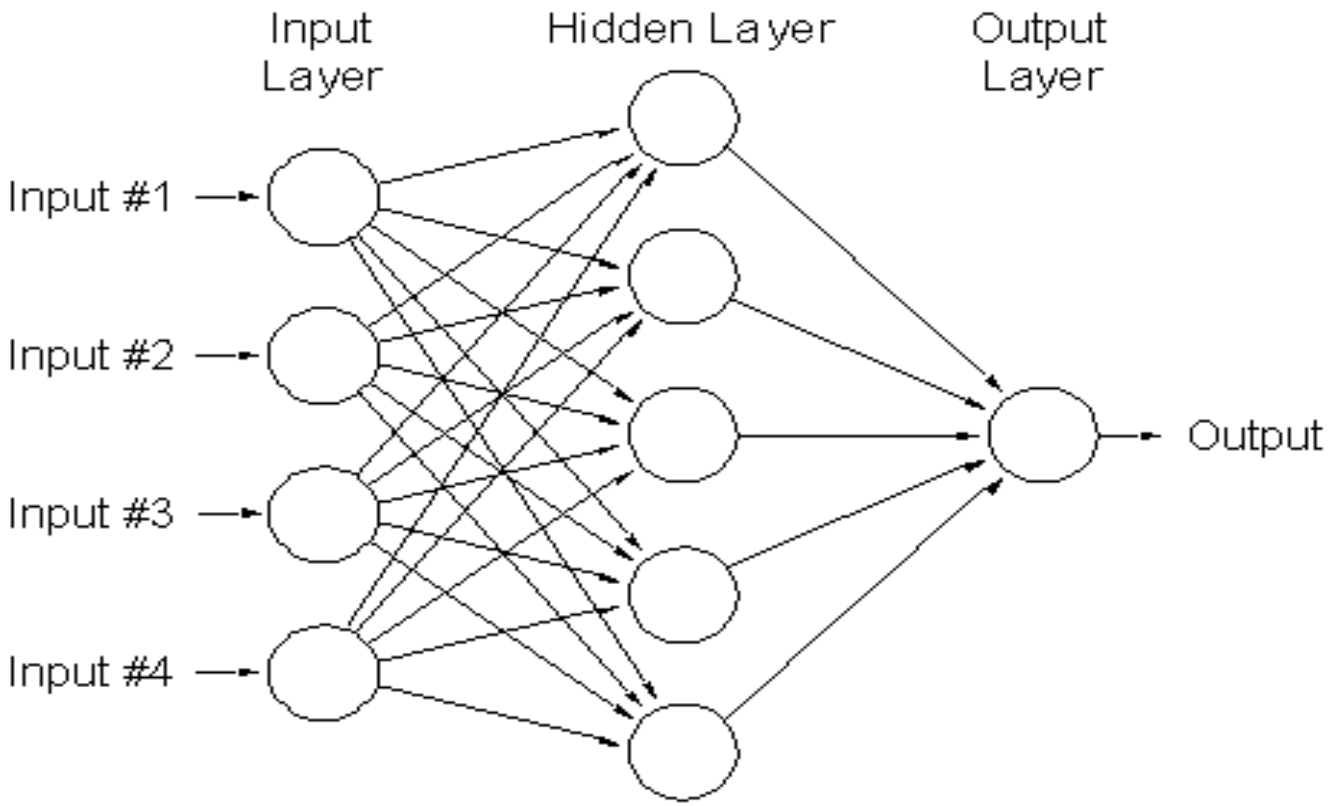
Introduction to Multilayer Perceptron(MLP)

A multilayer perceptron is a feedforward neural network with one or more hidden layers.

The network consists of:
-An input layer of source neurons,
-At least one middle or hidden layer of computational neurons,
-An output layer of computational neurons.

The input signals are propagated in a forward direction on a layer-by-layer basis.

Introduction to MLP



Introduction to MLP

Learning in MLP: Backpropagation

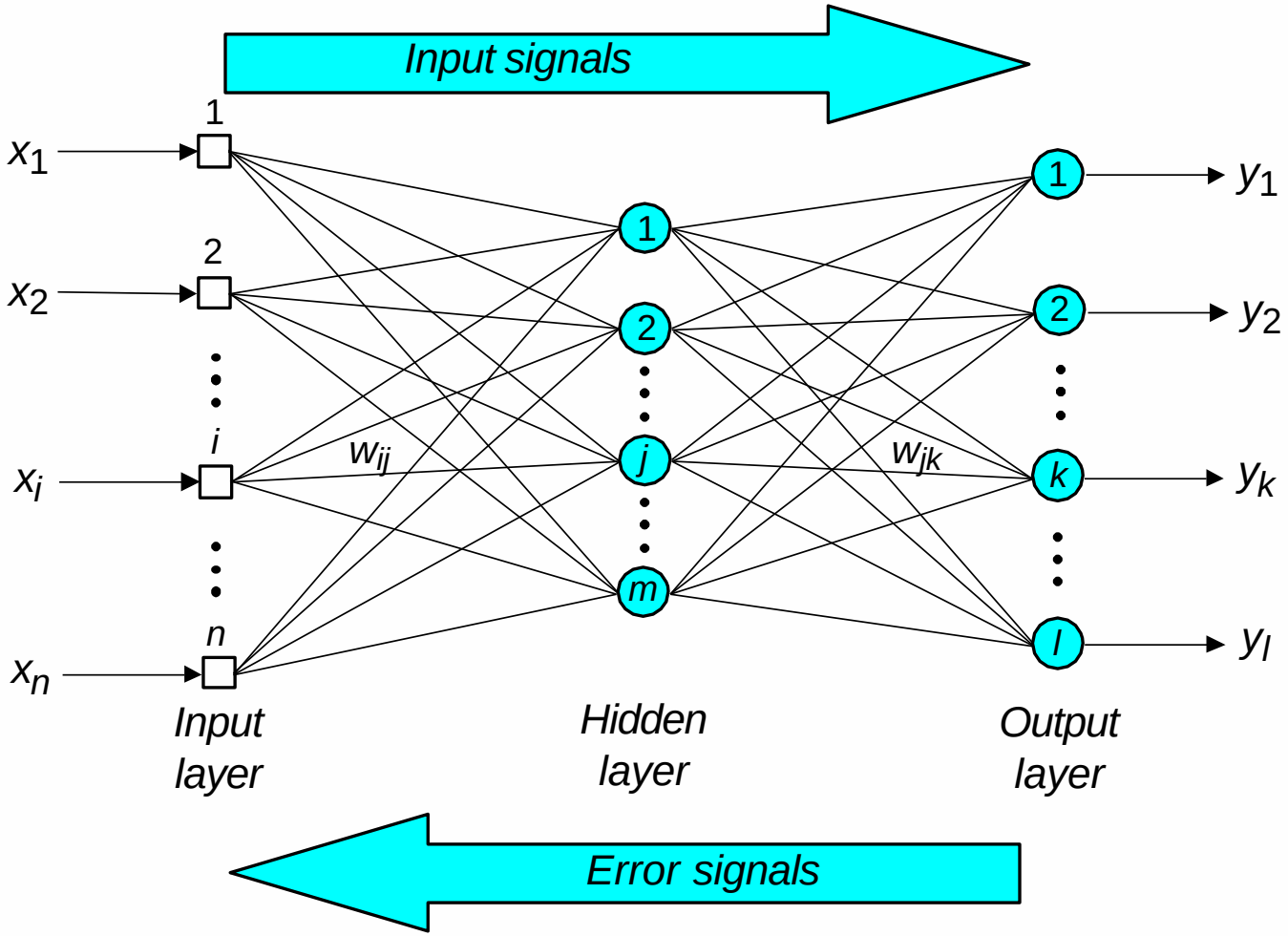
- Learning in a multilayer network proceeds the same way as for a perceptron.
- A training set of input patterns is presented to the network.
- The network computes its output pattern, and if there is an error - or in other words a difference between actual and desired output patterns - the weights are adjusted to reduce this error.

Introduction to MLP

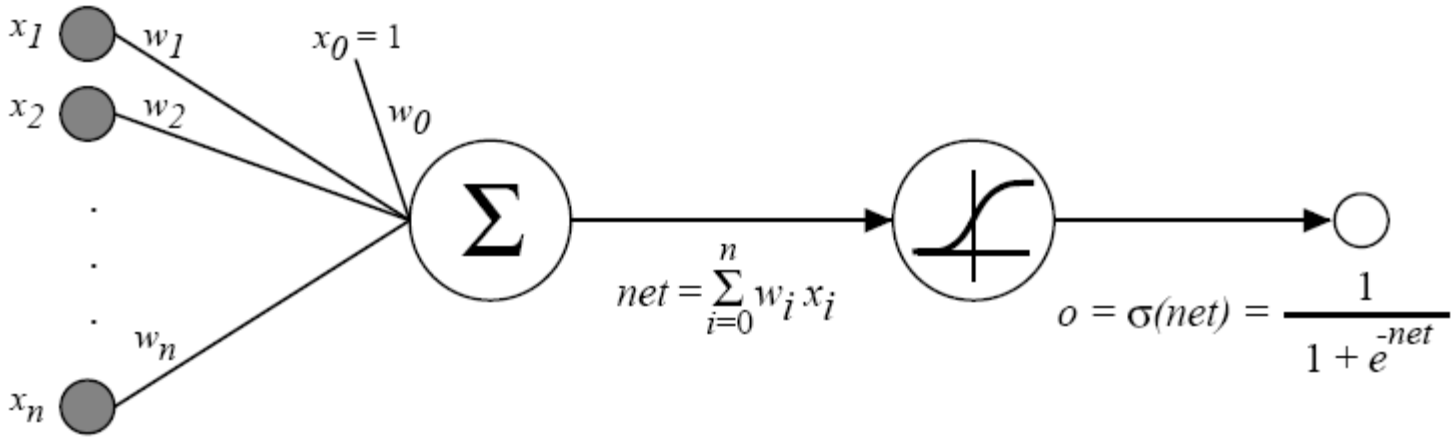
- In a back-propagation neural network, the learning algorithm has two phases.

1. A training input pattern is presented to the network input layer. The network propagates the input pattern from layer to layer until the output pattern is generated by the output layer.
2. If this pattern is different from the desired output, an error is calculated and then propagated backwards through the network from the output layer to the input layer. The weights are modified as the error is propagated.

Introduction to MLP



Activation Function

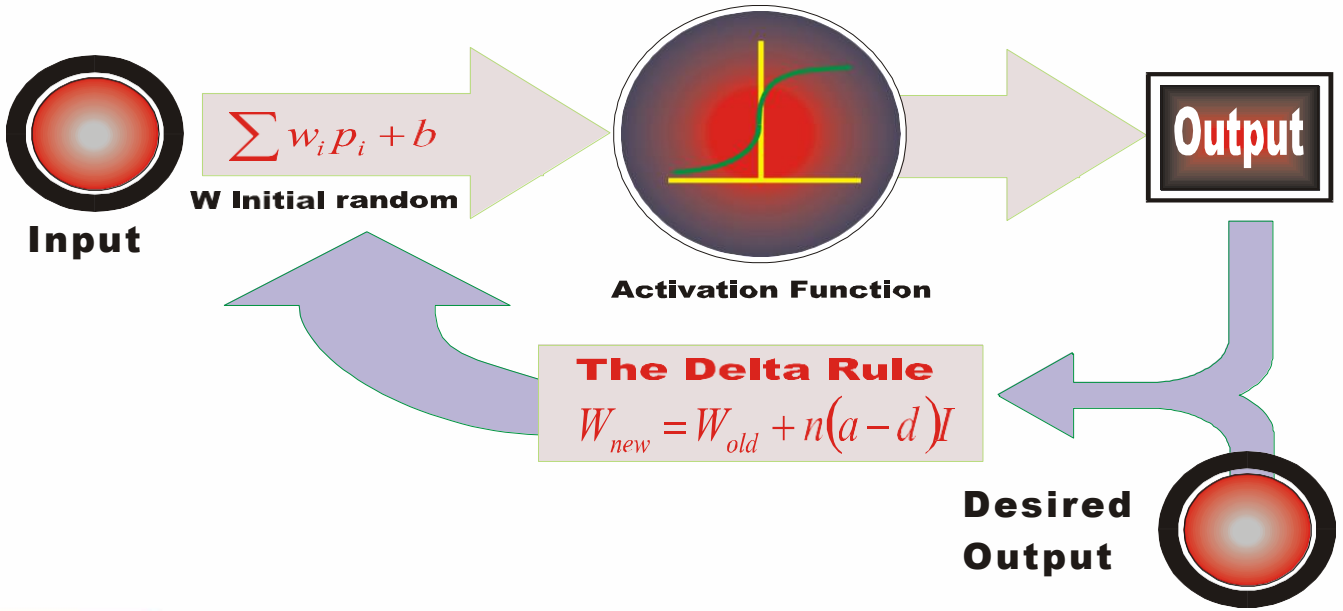


Sigmoid activation function: $f(x) = \frac{1}{1+e^{-x}}$

Delta Rules

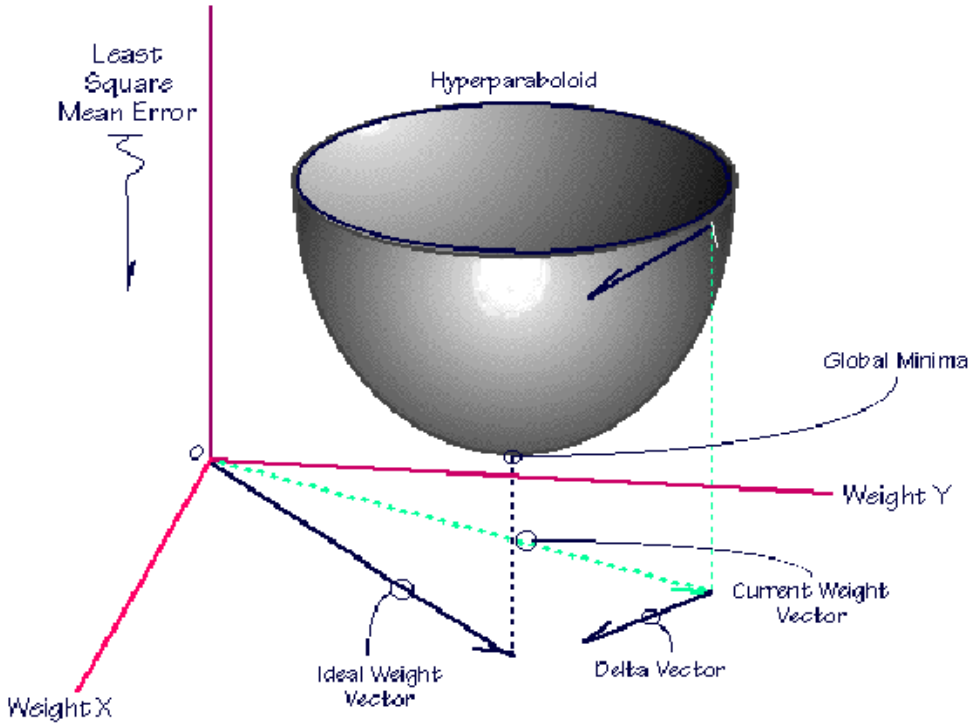
Often utilized by backpropagation

The network sees how far its answer was from the actual one and makes an appropriate adjustment to its connection weights.



Delta Rules

Backpropagation performs a gradient descent within the solution's vector space towards a global minimum and avoiding local minimum



Backpropagation Algorithm

1. Calculate the outputs of all neurons in the hidden layer:

$$x = \sum_{i=1}^{n} x_i \times w_i + bias$$

$$O_j = f(x) = \frac{1}{1 + e^{-x}}$$

2. Calculate the outputs of (all) neuron(s) in the output layer:

$$x = \sum_{i=1}^{n} x_i \times w_i + bias$$

$$O_k = f(x) = \frac{1}{1 + e^{-x}}$$

Backpropagation Algorithm

3. Calculate output error:

$$\delta_k = O_k \times (1 - O_k) \times (t - O_k)$$

4. Update weight between hidden-output layer:

$$\Delta w_{(jk)} = \alpha \times O_j \times \delta_k$$

$$w_{(jk)(t+1)} = w_{(jk)t} + \Delta w_{(jk)}$$

5. Calculate hidden error:

$$\delta_j = O_j \times (1 - O_j) \times \left(\sum_{k=1}^{n} \delta_k \times w_{(jk)}\right)$$

Backpropagation Algorithm

6. Update weight between input-hidden layer:

$$\Delta w_{(ij)} = \alpha \times x_i \times \delta_j$$

$$w_{(ij)(t+1)} = w_{(ij)t} + \Delta w_{(ij)}$$

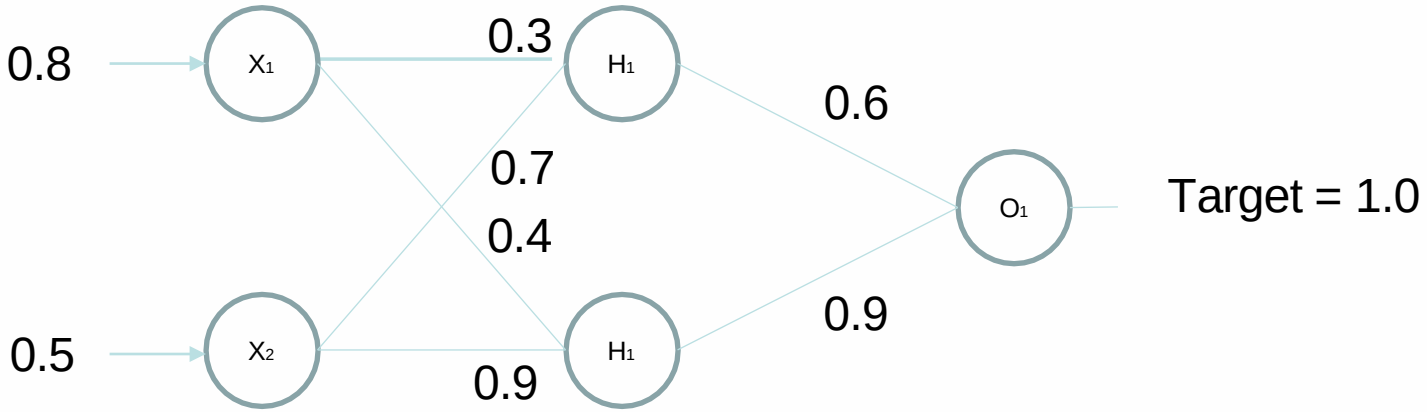
i = Input

j = Hidden

k = Output

Example:

Input (X)	Weight (Input to Hidden)	Weight (Hidden to Output)
$X_1 = 0.8$	X_1 to $H_1 = 0.3$	H_1 to $O_1 = 0.6$
$X_2 = 0.5$	X_1 to $H_2 = 0.4$	H_2 to $O_1 = 0.9$
	X_2 to $H_1 = 0.7$	
	X_2 to $H_2 = 0.9$	
Learning rate α = 0.6 **Target Output = 1.0**		



Example

1. Calculate the outputs of all neurons in the hidden layer:

   H_1 = (0.8 x 0.3) + (0.5 x 0.7) = 0.59, Sigmoid = 1/1+2.71828^-0.59 = 0.6434

   H_2 = (0.8 x 0.4) + (0.5 x 0.9) = 0.77, Sigmoid = 0.6835

2. Calculate the outputs of (all) neuron(s) in the output layer:

   O_1 = (0.6434 x 0.6) + (0.6835 x 0.9) = 1.0012, Sigmoid = 0.7313

3. Calculate output error

   Error = 0.7313 x (1-0.7313) x (1-0.7313) = 0.0528

Example

4. Update weight between hidden-output layer :

$W_{H1\text{-}O1} = 0.6 + (0.6 \times 0.6434 \times 0.0528) = 0.6204$

$W_{H2\text{-}O1} = 0.9 + (0.6 \times 0.6835 \times 0.0528) = 0.9217$

5. Calculate hidden error

6. Update weight between input-hidden layer:

Error gradient $H_1 = (0.0528 \times 0.6) \times ((1-0.6434) \times 0.6434) = 0.0073$

$W_{X1\text{-}H1} = 0.3 + (0.6 \times 0.8 \times 0.0073) = 0.3035$

$W_{X2\text{-}H1} = 0.7 + (0.6 \times 0.5 \times 0.0073) = 0.7021$

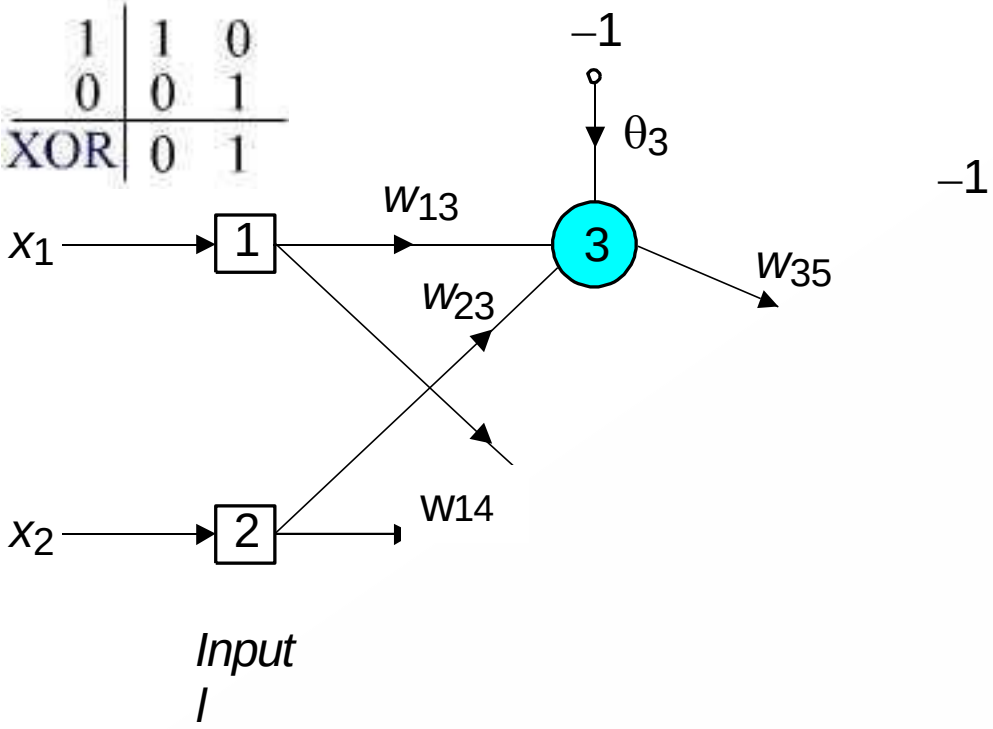
Error gradient $H_2 = (0.0528 \times 0.9) \times ((1-0.6835) \times 0.6835) = 0.0103$

$W_{X1\text{-}H2} = 0.4 + (0.6 \times 0.8 \times 0.0103) = 0.4049$

$W_{X2\text{-}H2} = 0.9 + (0.6 \times 0.5 \times 0.0103) = 0.9031$

Example: XOR Problem

1	1	0
0	0	1
XOR	0	1

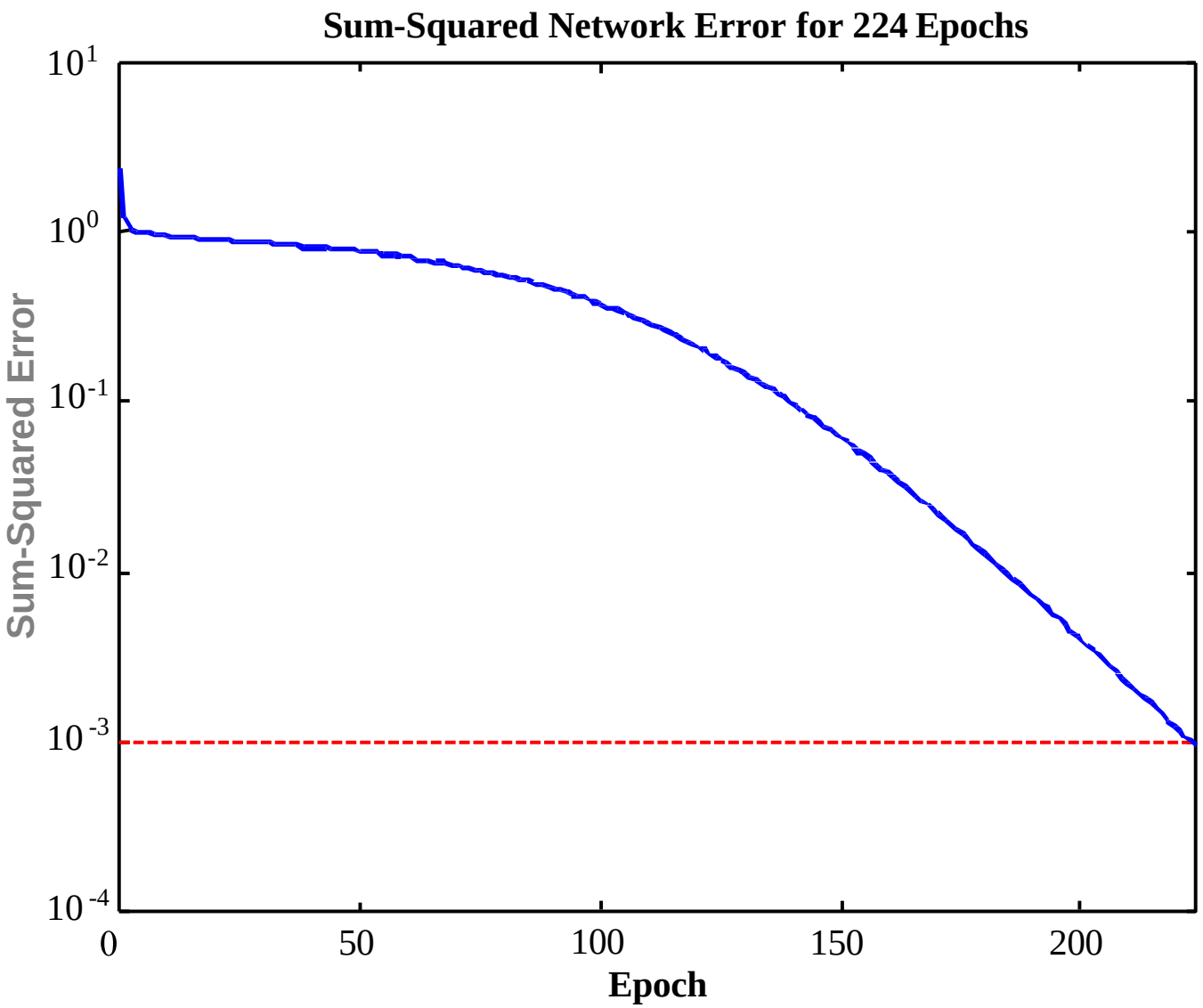


The initial weights and threshold levels are set randomly as follows:

w13 = 0.5, w14 = 0.9, w23 = 0.4, w24 = 1.0, w35 = -1.2, w45 = 1.1

θ3 = 0.8, θ 4 = -0.1 and θ 5 = 0.3.

Example: XOR Problem

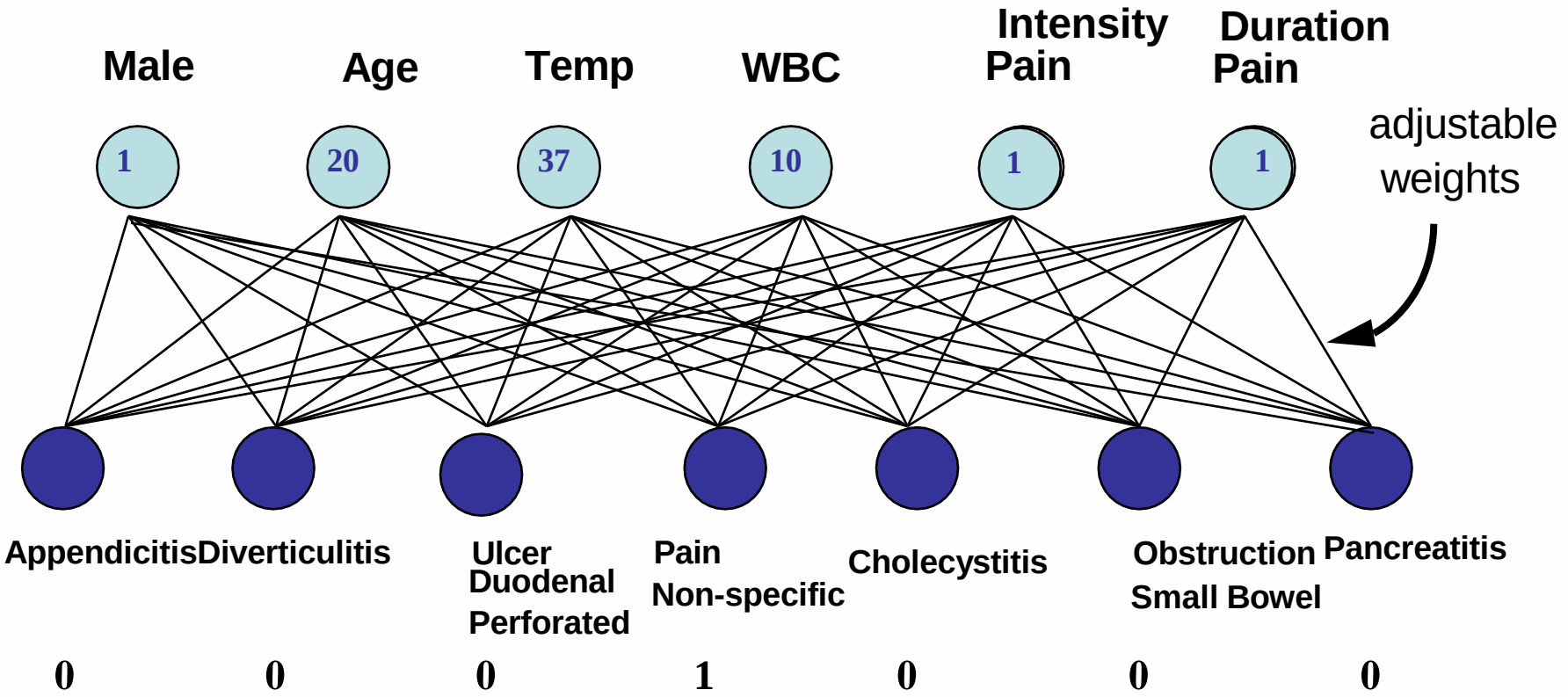


Example: XOR Problem

Inputs		Desired output	Actual output	Error	Sum of squared errors
x_1	x_2	y_d	y_5	e	
1	1	0	0.0155	–0.0155	0.0010
0	1	1	0.9849	0.0151	
1	0	1	0.9849	0.0151	
0	0	0	0.0175	–0.0175	

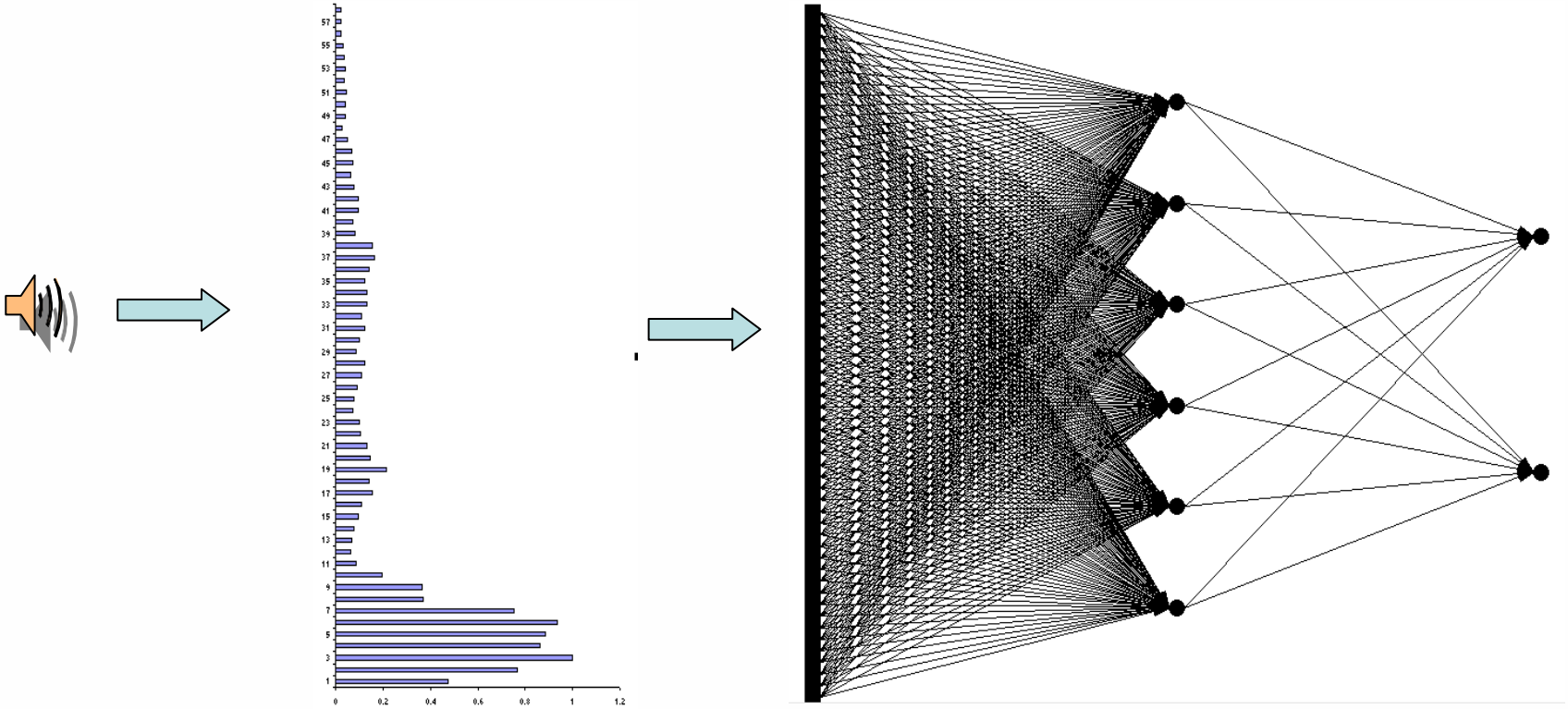
Applications

Abdominal Pain Prediction



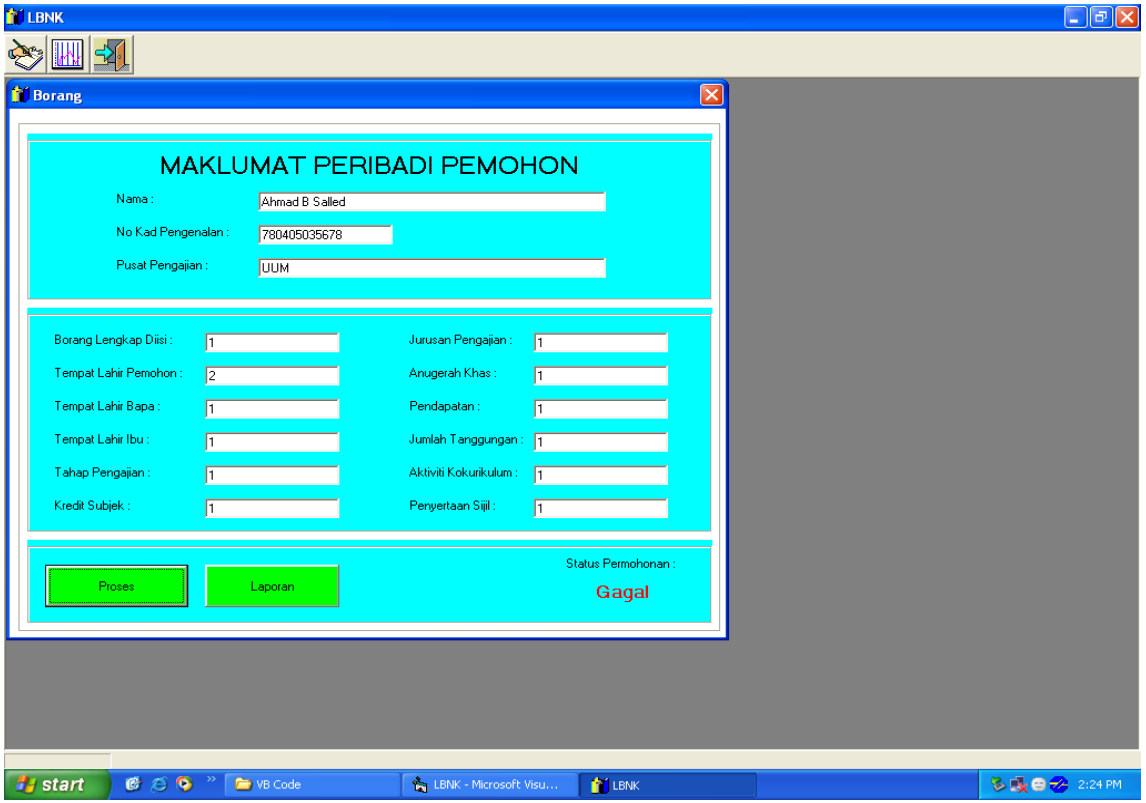
Applications

Voice Recognition



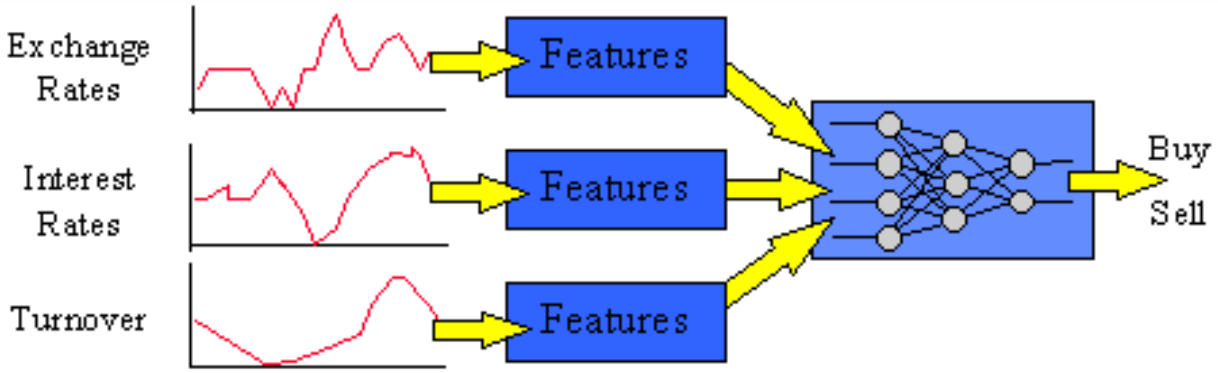
Applications

Educational Loan Forecasting System



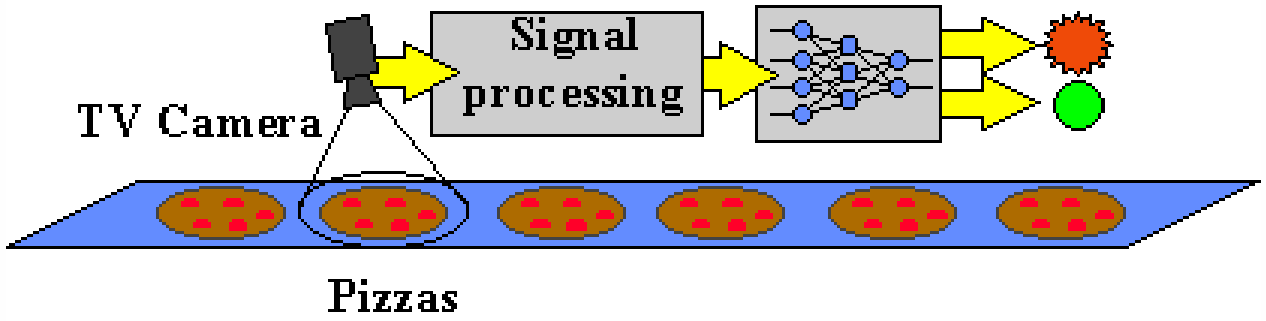
Applications

Stock Market Prediction



Applications

Automated Industrial Inspection



Traditional vs ANN

CHARACTERISTICS	TRADITIONAL COMPUTING (including Expert Systems)	ARTIFICIAL NEURAL NETWORKS
Processing style	Sequential	Parallel
Functions	Logically (left brained) via Rules Concepts Calculations	Gestault (right brained) via Images Pictures Controls
Learning Method	by rules (didactically)	by example (Socratically)
Applications	Accounting, word processing, math, inventory, digital communications	Sensor processing, speech recognition, pattern recognition, text recognition

Lab Exercise 2

Download and install WEKA

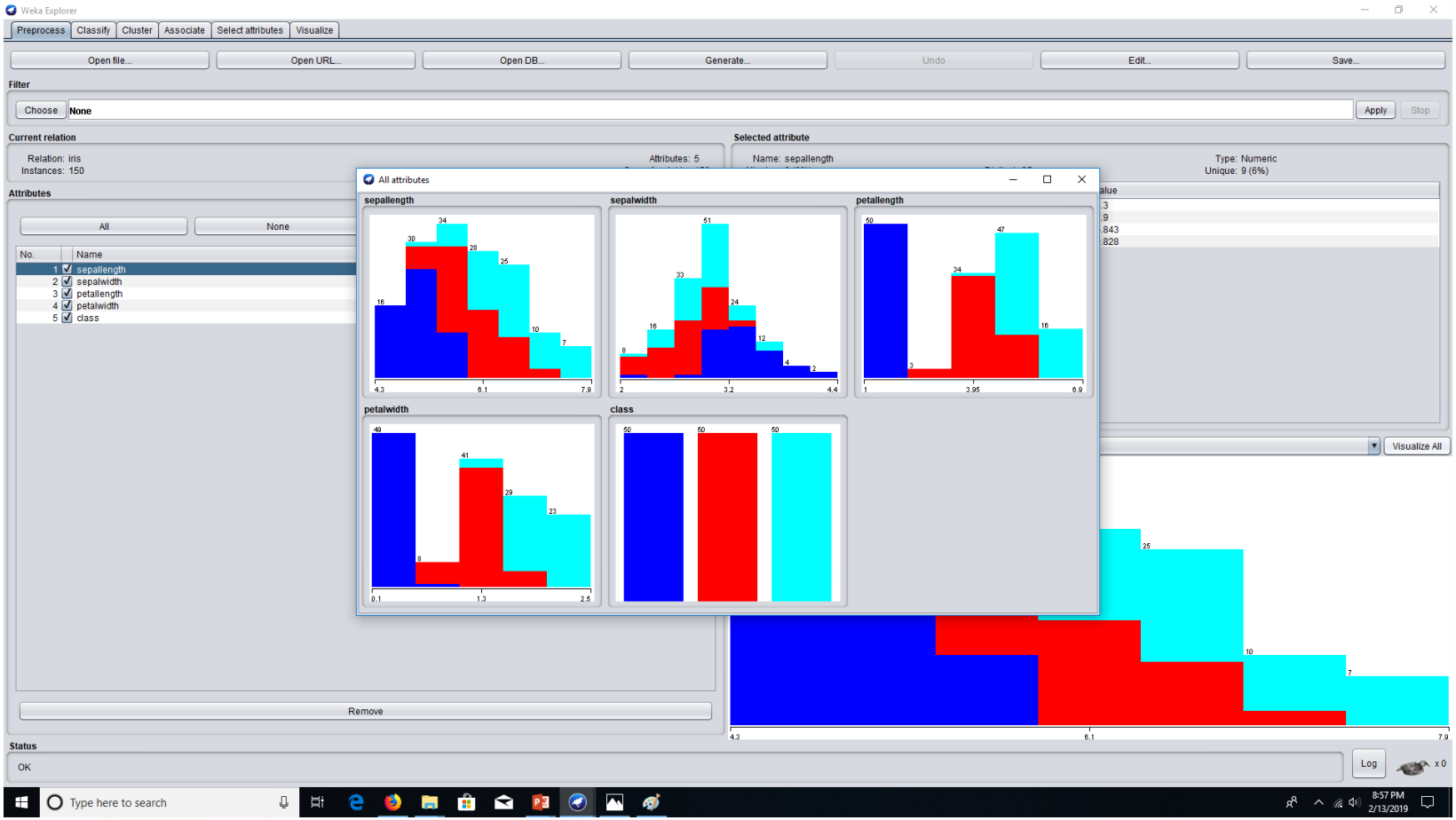
https://www.cs.waikato.ac.nz/ml/weka/

Download dataset from UCI Machine Learning and prepare ARFF format

https://www.cs.waikato.ac.nz/ml/weka/arff.html

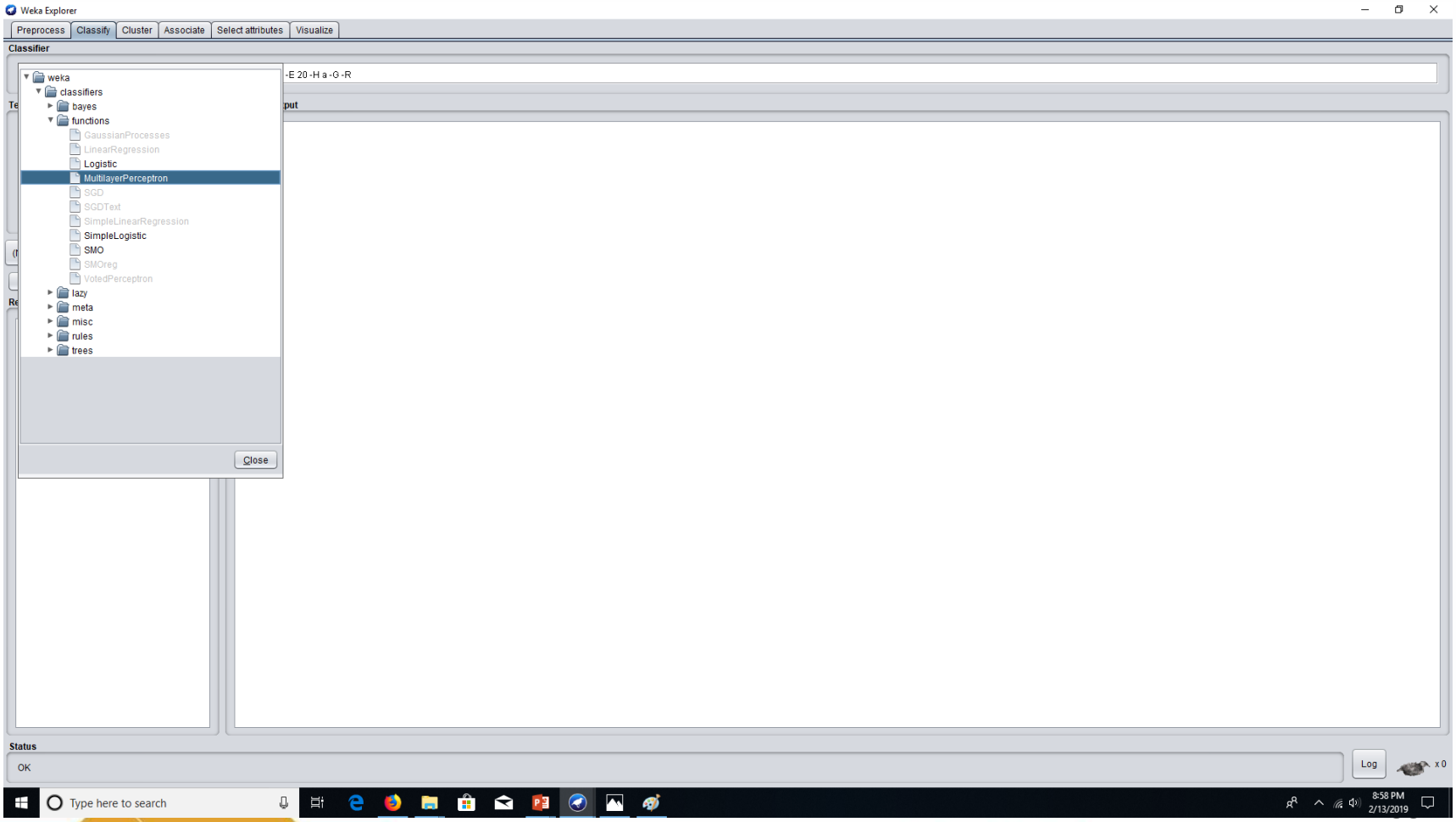
Lab Exercise 2

Data visualization



Lab Exercise 2

Classification using MLP



Lab Exercise 2

